

AN AGENTIC AI ENGINEERING FIELD GUIDE

# The AI Engineer's Production *Checklist*.

From prototype to shipped. The senior-level checklist for putting LLMs, RAG, agents, and evals into production, with the one gate that tells you each layer is actually done.

FORMAT

Checklist

STAGES

5

READ TIME

12 min

---

By Aki Wijesundara, PhD & Manu Jayawardana

Founders, Snapdrum · Instructors at Maven

# The demo works.

## *Shipping is another discipline.*

A notebook that answers correctly once is a demo. A system that answers correctly on the inputs you have not seen yet, under load, within budget, and without leaking, is a product. This checklist is the distance between the two.

Most AI projects die in that gap, and the model is rarely the reason. The failure is almost always in the plumbing around it: an unvalidated output that crashes a downstream service, a retrieval step nobody measured, an agent that loops forever, a prompt change that silently regressed and shipped on a Friday.

The five stages below map to the way a production AI system is actually built, roughly in the order the failures tend to arrive. Each stage carries a checklist you can run against your own system today, plus the single gate that means the stage is finished.

**How to use this:** keep it open while you build. Run each stage's checklist before you call that layer done. When something breaks in production, come back and find the check you skipped. It is almost always one of these.

| NO. | STAGE                          | THE GATE THAT MEANS DONE                                                 |
|-----|--------------------------------|--------------------------------------------------------------------------|
| 01  | The model behind an API        | Outputs are schema-validated and prompts are versioned and pinned.       |
| 02  | Retrieval that earns its place | Retrieval quality is measured before any generation prompt is tuned.     |
| 03  | Agents that stay in bounds     | Step caps, least-privilege tools, human sign-off on destructive actions. |
| 04  | Evals, the TRACE loop          | A regression set runs in CI and gates every deploy.                      |
| 05  | Memory and shipping            | Runs are traceable, resumable, and reversible, with a kill switch.       |

# Put the model behind a real *API*.

The first job is turning a prompt that works in a playground into an endpoint a teammate can call a thousand times without surprises.

---

## Prompting that *ships*

- ◆ **VERSIONED** Prompts live in the repo as versioned templates, not inline strings scattered across the codebase. A prompt change is a reviewable diff.
- ◆ **ROLES** System, developer, and user content stay in separate channels. Instructions are never concatenated into user-supplied text.
- ◆ **PINNED** The model version is pinned. No floating alias quietly changes your behavior overnight.

---

## Structured output and *guardrails*

- ◆ **SCHEMA** Outputs use the provider's structured-output or tool-calling mode and validate against a schema (Pydantic, zod). You never parse free text with a regex.
- ◆ **REPAIR** A parse failure triggers one repair retry, then a typed fallback. A malformed response never takes down the request.
- ◆ **BOUNDARIES** Inputs are length-capped and screened. Outputs are checked for PII and unsafe content before a user ever sees them.

---

## Cost and model *choice*

- ◆ **BUDGET** `max_tokens` is set, the context window is budgeted on purpose, and stable prompt prefixes are cached.
- ◆ **RIGHT-SIZE** The smallest model that passes your evals serves the request, with a fallback model for provider outages. You measure cost per successful task, not per token.

# Make RAG *earn* its place.

Retrieval is the default answer to “the model does not know our data,” and it is wrong about as often as it is right. Measure it, or skip it.

---

## Build the index *well*

- ◆ **CHUNK** Chunks follow semantic boundaries (headings, paragraphs) with light overlap, not fixed character counts. Each chunk carries metadata: source, section, date.
- ◆ **HYBRID** Retrieval blends dense embeddings with sparse keyword search (BM25), fused together, not one alone.
- ◆ **RERANK** You retrieve wide (top 50 to 100), then a cross-encoder reranker narrows to the handful the model actually reads.

---

## Prove it *works*

- ◆ **MEASURE** Retrieval quality is measured on a labeled set (recall@k, MRR) before a single generation prompt is tuned. Bad retrieval cannot be fixed downstream.

### REACH FOR RAG

**The data is large, private, and citable.**

- The corpus is large, changing, or private
- Answers live in specific passages you can cite
- Freshness matters and the index updates
- Grounding and citations are a hard requirement

### SKIP RAG

**Context, SQL, or a tool fits better.**

- The corpus is small and stable (put it in context)
- The answer needs the whole document, not snippets
- The data is structured (query it with SQL or a tool)
- You have not yet shown retrieval helps at all

# Give the agent a loop, and a *leash*.

An agent is a model in a loop with tools: it decides, acts, observes, and repeats until done. Production is mostly about what happens when “done” never arrives, or when a tool does something it should not.

---

## The *loop*

- ◆ **CAP** Every run has a hard ceiling on steps, tokens, time, and cost. An agent that cannot finish stops cleanly, it does not spin.
- ◆ **OBSERVE** Each tool result returns structured and concise. The model reasons over data, not over raw dumps it has to re-parse.

---

## Tools via *MCP*

- ◆ **DESCRIBE** Tool schemas and descriptions are tight. The model knows only what the schema says, so the schema is the contract. Exposing tools over MCP keeps them portable across hosts.
- ◆ **VALIDATE** Tool inputs are validated and tools are idempotent where they can be. A retried call does not double-charge or double-send.

---

## Staying in *bounds*

- ◆ **UNTRUSTED** All retrieved, tool, and user content is treated as untrusted. Retrieved text can never issue privileged actions. The instruction channel is separate from the data channel.
- ◆ **LEAST PRIVILEGE** Each tool holds the narrowest permission that works, runs in a sandbox where it can, and destructive or irreversible actions require a human in the loop.

### SINGLE AGENT FIRST

Reach for one capable agent with good tools before you reach for a crowd. Multi-agent orchestration earns its complexity only when the work genuinely splits into separable, parallel tasks with clean handoffs. **Most “multi-agent” systems are one agent with a context-management problem.**

# Close the *TRACE* loop.

You cannot improve what you cannot see, and you cannot ship safely on a number you never wrote down. TRACE is the loop that turns real failures into permanent guardrails.

---

|          |                |                                                                                                                                              |
|----------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>T</b> | <b>Trace</b>   | Instrument everything. Full inputs, outputs, tool calls, and intermediate steps are logged. <b>You cannot eval what you did not capture.</b> |
| <b>R</b> | <b>Read</b>    | Read the traces by hand. Real failures, not aggregate scores. Error analysis comes before dashboards.                                        |
| <b>A</b> | <b>Analyze</b> | Cluster failures into modes and quantify them. Fix the dominant mode, not the loudest anecdote.                                              |
| <b>C</b> | <b>Codify</b>  | Turn each failure mode into an eval case. Code checks for hard constraints, an LLM judge with a rubric for the fuzzy ones.                   |
| <b>E</b> | <b>Enforce</b> | Run the eval set in CI and gate every deploy on it. A change that regresses the set does not merge.                                          |

---

## Two evals people *skip*

- ◆ **JUDGE** Your LLM-as-judge scores against a written rubric and is calibrated to human labels on a sample. An uncalibrated judge is a vibe with an API bill.
- ◆ **TRAJECTORY** Agents are evaluated on the path, not just the final answer. The right answer reached by the wrong route is still a bug waiting to surface.

# Remember only what you *must*, then ship it.

Memory is a spectrum, not a feature you bolt on. Pick the least memory the task needs, then put the whole thing behind a runtime that survives a restart.

| LEVEL      | WHAT IT REMEMBERS          | REACH FOR IT WHEN                                              |
|------------|----------------------------|----------------------------------------------------------------|
| Stateless  | Nothing between calls      | One-shot tasks: extraction, classification, a single answer.   |
| Buffer     | The recent turns           | Short conversations where only the last few messages matter.   |
| Summary    | A compressed history       | Long sessions where the gist matters more than the transcript. |
| Retrieval  | Past facts on demand       | A large history you query like a corpus (RAG over memory).     |
| Persistent | Structured facts, entities | Agents that must recall users and state across sessions.       |

## Before it goes *live*

- ◆ **DURABLE** Long runs use a durable runtime: state is checkpointed, runs are resumable, retries are idempotent. A restart does not lose the job.
- ◆ **OBSERVE** The same tracing you built for evals runs in production, with cost and latency monitoring and alerts on spikes.
- ◆ **REVERSIBLE** Prompts and model versions are pinned so you can roll back, rollout sits behind a flag, and there is a kill switch.

# Ship-readiness, *one page.*

| NO. | STAGE                   | THE GATE THAT MEANS READY                                                                                   |
|-----|-------------------------|-------------------------------------------------------------------------------------------------------------|
| 01  | The model behind an API | Structured outputs validated, prompts versioned and pinned, cost per task tracked.                          |
| 02  | Retrieval               | Recall@k measured on a labeled set before any generation tuning. Hybrid search, reranked.                   |
| 03  | Agents                  | Step and cost caps, least-privilege tools, untrusted-input boundary, human sign-off on destructive actions. |
| 04  | Evals                   | TRACE closed: a regression set runs in CI and gates every deploy. Judge calibrated, trajectory checked.     |
| 05  | Memory and shipping     | Least memory that works, durable runtime, traceable and resumable runs, kill switch and rollback.           |

## RED FLAGS: YOU ARE NOT READY IF

A malformed model response can crash a request. Nobody has measured retrieval quality. An agent can take a destructive action without a human. There is no eval that would catch the regression you are about to ship. You cannot roll back the last prompt change in under a minute.

**The pattern underneath all five:** every stage is the same move applied to a different layer. Make the behavior observable, write down what good looks like, then put a gate in front of the thing that bites. Demos skip the gates. Products live and die by them.

# Run this against *your* build.

Print this, or paste it into your pull-request template. Tick a box only when it is true in code, not in intention.

---

## The model *layer*

☐ **OUTPUT** Every model output is schema-validated, with a repair retry and a typed fallback.

☐ **COST** I can state the cost per successful task and the model I would fall back to.

---

## Retrieval and *agents*

☐ **RETRIEVAL** I have a recall@k number for my retriever on a labeled set, not a guess.

☐ **LEASH** Every agent run has a step, time, and cost cap, and destructive actions need sign-off.

---

## Evals and *shipping*

☐ **REGRESSION** A failure I have actually seen in a trace is now an eval case that runs in CI.

☐ **ROLLBACK** I can roll back the current prompt and model in under a minute, and there is a kill switch.

BUILD THE WHOLE STACK WITH US

### Agentic AI Engineering Bootcamp · 4 Weeks

Live with Aki and Manu. Build production AI agents, RAG, and evals, then ship on Demo Day. Office hours, recordings, lifetime access.

ENROLL